

Go One-Stop Regular Problems and Solutions

Collections-like structures (slices/maps/heaps), channels, goroutines, caching, and core DSA/Algo patterns.

Section A: Core Data Handling (G1-G8)

G1. Two Sum with map

```
func twoSum(nums []int, target int) []int {
    seen := map[int]int{}
    for i, n := range nums {
        if j, ok := seen[target-n]; ok {
            return []int{j, i}
        }
        seen[n] = i
    }
    return []int{-1, -1}
}
```

G2. Group Anagrams

```
func groupAnagrams(strs []string) [][]string {
    m := map[string][]string{}
    for _, s := range strs {
        b := []byte(s)
        slices.Sort(b)
        k := string(b)
        m[k] = append(m[k], s)
    }
    out := make([][]string, 0, len(m))
    for _, v := range m { out = append(out, v) }
    return out
}
```

G3. Top K Frequent using heap

```
import "container/heap"

type pair struct{ val, freq int }
type minHeap []pair
func (h minHeap) Len() int { return len(h) }
func (h minHeap) Less(i, j int) bool { return h[i].freq < h[j].freq }
func (h minHeap) Swap(i, j int) { h[i], h[j] = h[j], h[i] }
func (h *minHeap) Push(x any) { *h = append(*h, x.(pair)) }
func (h *minHeap) Pop() any { n := len(*h); x := (*h)[n-1]; *h = (*h)[:n-1]; return x }
```

G4. Merge Intervals

```
func merge(intervals [][]int) [][]int {
    slices.SortFunc(intervals, func(a, b []int) int { return a[0] - b[0] })
    out := make([][]int, 0, len(intervals))
    for _, cur := range intervals {
        if len(out) == 0 || out[len(out)-1][1] < cur[0] {
            out = append(out, []int{cur[0], cur[1]})
        } else if cur[1] > out[len(out)-1][1] {
            out[len(out)-1][1] = cur[1]
        }
    }
    return out
}
```

G5. Sliding Window Maximum

```
func maxSlidingWindow(nums []int, k int) []int {
    dq := []int{}
    ans := []int{}
    for i, x := range nums {
        for len(dq) > 0 && dq[0] <= i-k { dq = dq[1:] }
        for len(dq) > 0 && nums[dq[len(dq)-1]] <= x { dq = dq[:len(dq)-1] }
        dq = append(dq, i)
        if i >= k-1 { ans = append(ans, nums[dq[0]]) }
    }
    return ans
}
```

G6. LRU via list + map (pattern)

- Use `map[key]*list.Element` + `container/list`.
- Move on hit to front; evict from back when size exceeds cap.

G7. Word Frequency

```
func freq(words []string) map[string]int {
    m := map[string]int{}
    for _, w := range words { m[w]++ }
    return m
}
```

G8. Deep copy slice

```
func clone(a []int) []int { return append([]int(nil), a...) }
```

Section B: Goroutines, Channels, and Sync (G9-G16)

G9. Fan-out worker pool

```
func worker(jobs <-chan int, out chan<- int, wg *sync.WaitGroup) {
    defer wg.Done()
    for j := range jobs { out <- j * j }
}
```

G10. Context cancellation

```
func do(ctx context.Context) error {
    select {
    case <-time.After(100 * time.Millisecond):
        return nil
    case <-ctx.Done():
        return ctx.Err()
    }
}
```

G11. Rate limit with ticker

```
limiter := time.NewTicker(50 * time.Millisecond)
defer limiter.Stop()
for range requests {
    <-limiter.C
}
```

G12. Safe counter (mutex)

```
type Counter struct { mu sync.Mutex; n int }
func (c *Counter) Inc() { c.mu.Lock(); c.n++; c.mu.Unlock() }
```

G13. Safe counter (atomic)

```
var n int64
atomic.AddInt64(&n, 1)
```

G14. WaitGroup usage

```
var wg sync.WaitGroup
wg.Add(1)
go func(){ defer wg.Done() }()
wg.Wait()
```

G15. Buffered channel backpressure

```
ch := make(chan int, 100)
```

Use bounded buffer to avoid unbounded memory growth.

G16. Select default for non-blocking send

```
select {
case ch <- v:
default:
    // drop or retry
}
```

Section C: Cache and Production Patterns (G17-G22)

G17. Read-through cache

```
type Cache struct { mu sync.RWMutex; m map[string]string }
func (c *Cache) Get(k string, load func(string) string) string {
    c.mu.RLock(); v, ok := c.m[k]; c.mu.RUnlock()
    if ok { return v }
    v = load(k)
    c.mu.Lock(); if c.m == nil { c.m = map[string]string{} }; c.m[k] = v; c.mu.Unlock()
    return v
}
```

G18. TTL cache entry

```
type entry struct { val string; exp time.Time }
```

Store expiry and check on read.

G19. Singleflight to prevent stampede

```
var g singleflight.Group
v, err, _ := g.Do(key, func() (any, error) { return load(key), nil })
```

G20. Negative caching

Store not-found sentinel with short TTL.

G21. Sharded map

Split map into shards by hash to reduce lock contention.

G22. Run race detector

```
go test -race ./...
```

Section D: DSA/Algo Core (G23-G35)

G23. Binary search

```
func binarySearch(a []int, target int) int {
    l, r := 0, len(a)-1
    for l <= r {
        m := l + (r-l)/2
        if a[m] == target {
            return m
        }
        if a[m] < target {
            l = m + 1
        } else {
            r = m - 1
        }
    }
    return -1
}
```

G24. BFS shortest path

```
func shortestPath(g [][]int, src, dst int) int {
    d := make([]int, len(g))
    for i := range d {
        d[i] = -1
    }
    q := []int{src}
    d[src] = 0

    for len(q) > 0 {
        u := q[0]
        q = q[1:]
        if u == dst {
            return d[u]
        }
        for _, v := range g[u] {
            if d[v] == -1 {
                d[v] = d[u] + 1
                q = append(q, v)
            }
        }
    }
    return -1
}
```

G25. DFS cycle detection

```
func hasCycle(g [][]int) bool {
    state := make([]int, len(g))
    var dfs func(int) bool
    dfs = func(u int) bool {
        if state[u] == 1 {
            return true
        }
        if state[u] == 2 {
            return false
        }
        state[u] = 1
        for _, v := range g[u] {
            if dfs(v) {
                return true
            }
        }
        state[u] = 2
        return false
    }
    for i := range g {
        if dfs(i) {
            return true
        }
    }
    return false
}
```

G26. Topological sort (Kahn)

```
func topoSort(n int, edges [][]int) []int {
    g := make([][]int, n)
    indeg := make([]int, n)
    for _, e := range edges {
        u, v := e[0], e[1]
        g[u] = append(g[u], v)
        indeg[v]++
    }
    q := []int{}
    for i := 0; i < n; i++ {
        if indeg[i] == 0 {
            q = append(q, i)
        }
    }
    out := []int{}
    for len(q) > 0 {
        u := q[0]
        q = q[1:]
        out = append(out, u)
    }
}
```

```

    for _, v := range g[u] {
        indeg[v]--
        if indeg[v] == 0 {
            q = append(q, v)
        }
    }
}
if len(out) != n {
    return nil
}
return out
}

```

G27. Dijkstra with heap

```

type node struct{ v, d int }
type minQ []node

func (h minQ) Len() int { return len(h) }
func (h minQ) Less(i, j int) bool { return h[i].d < h[j].d }
func (h minQ) Swap(i, j int) { h[i], h[j] = h[j], h[i] }
func (h *minQ) Push(x any) { *h = append(*h, x.(node)) }
func (h *minQ) Pop() any { n := len(*h); x := (*h)[n-1]; *h = (*h)[:n-1]; return x }

func dijkstra(g [][][2]int, src int) []int {
    const inf = int(1e9)
    dist := make([]int, len(g))
    for i := range dist {
        dist[i] = inf
    }
    dist[src] = 0
    pq := &minQ{{src, 0}}
    heap.Init(pq)

    for pq.Len() > 0 {
        cur := heap.Pop(pq).(node)
        if cur.d != dist[cur.v] {
            continue
        }
        for _, e := range g[cur.v] {
            to, w := e[0], e[1]
            if dist[cur.v]+w < dist[to] {
                dist[to] = dist[cur.v] + w
                heap.Push(pq, node{to, dist[to]})
            }
        }
    }
    return dist
}

```

G28. Union-Find

```
type DSU struct {
    p, r []int
}

func newDSU(n int) *DSU {
    p := make([]int, n)
    r := make([]int, n)
    for i := 0; i < n; i++ {
        p[i] = i
    }
    return &DSU{p: p, r: r}
}

func (d *DSU) find(x int) int {
    if d.p[x] != x {
        d.p[x] = d.find(d.p[x])
    }
    return d.p[x]
}

func (d *DSU) union(a, b int) bool {
    ra, rb := d.find(a), d.find(b)
    if ra == rb {
        return false
    }
    if d.r[ra] < d.r[rb] {
        d.p[ra] = rb
    } else if d.r[ra] > d.r[rb] {
        d.p[rb] = ra
    } else {
        d.p[rb] = ra
        d.r[ra]++
    }
    return true
}
```

G29. Kadane max subarray

```
func maxSubArray(a []int) int {
    best, cur := a[0], a[0]
    for i := 1; i < len(a); i++ {
        if cur < 0 {
            cur = a[i]
        } else {
            cur += a[i]
        }
        if cur > best {
            best = cur
        }
    }
    return best
}
```

```

    }
}
return best
}

```

G30. Prefix sum subarray-k

```

func subarraySum(nums []int, k int) int {
    cnt := map[int]int{0: 1}
    sum, ans := 0, 0
    for _, x := range nums {
        sum += x
        ans += cnt[sum-k]
        cnt[sum]++
    }
    return ans
}

```

G31. LIS $O(n \log n)$

```

func lis(nums []int) int {
    tails := []int{}
    for _, x := range nums {
        i, j := 0, len(tails)
        for i < j {
            m := i + (j-i)/2
            if tails[m] < x {
                i = m + 1
            } else {
                j = m
            }
        }
        if i == len(tails) {
            tails = append(tails, x)
        } else {
            tails[i] = x
        }
    }
    return len(tails)
}

```

G32. Coin change DP

```

func coinChange(coins []int, amount int) int {
    const inf = int(1e9)
    dp := make([]int, amount+1)
    for i := 1; i <= amount; i++ {
        dp[i] = inf
    }
}

```

```

    }
    for a := 1; a <= amount; a++ {
        for _, c := range coins {
            if a-c >= 0 && dp[a-c]+1 < dp[a] {
                dp[a] = dp[a-c] + 1
            }
        }
    }
    if dp[amount] >= inf {
        return -1
    }
    return dp[amount]
}

```

G33. Knapsack DP

```

func knapsack(w, v []int, cap int) int {
    dp := make([]int, cap+1)
    for i := 0; i < len(w); i++ {
        for c := cap; c >= w[i]; c-- {
            if dp[c-w[i]]+v[i] > dp[c] {
                dp[c] = dp[c-w[i]] + v[i]
            }
        }
    }
    return dp[cap]
}

```

G34. Trie insert/search

```

type trieNode struct {
    next [26]*trieNode
    end bool
}

type Trie struct { root *trieNode }

func newTrie() *Trie { return &Trie{root: &trieNode{}} }

func (t *Trie) Insert(s string) {
    cur := t.root
    for _, ch := range s {
        i := ch - 'a'
        if cur.next[i] == nil {
            cur.next[i] = &trieNode{}
        }
        cur = cur.next[i]
    }
    cur.end = true
}

```

```

}

func (t *Trie) Search(s string) bool {
    cur := t.root
    for _, ch := range s {
        i := ch - 'a'
        if cur.next[i] == nil {
            return false
        }
        cur = cur.next[i]
    }
    return cur.end
}

```

G35. Backtracking subsets/permutations

```

func subsets(nums []int) [][]int {
    out := [][]int{}
    cur := []int{}
    var dfs func(int)
    dfs = func(i int) {
        if i == len(nums) {
            copyCur := append([]int(nil), cur...)
            out = append(out, copyCur)
            return
        }
        dfs(i + 1)
        cur = append(cur, nums[i])
        dfs(i + 1)
        cur = cur[:len(cur)-1]
    }
    dfs(0)
    return out
}

```

Section E: Advanced Production Patterns (G36-G45)

G36. errgroup for concurrent requests

```

g, ctx := errgroup.WithContext(context.Background())
for _, url := range urls {
    u := url
    g.Go(func() error { return fetch(ctx, u) })
}
if err := g.Wait(); err != nil { return err }

```

G37. HTTP client with timeout and retry budget

```
client := &http.Client{Timeout: 2 * time.Second}
```

Use capped retries and stop on context cancellation.

G38. Circuit breaker interview pattern

- CLOSED: normal calls
- OPEN: fail fast until cooldown
- HALF-OPEN: limited probes

G39. Idempotency key handling

Persist request ID and response hash to prevent duplicate side effects.

G40. Graceful shutdown with signal

```
ctx, stop := signal.NotifyContext(context.Background(), os.Interrupt, syscall.SIGTERM)
defer stop()
```

G41. Bounded worker pool with cancellation

Use jobs channel, fixed workers, and close on context done.

G42. Token bucket rate limiter

Use golang.org/x/time/rate for external API protection.

G43. Observability baseline

- structured logs with request ID
- RED metrics (rate, errors, duration)
- p95 and p99 latency alerts

G44. Data race debugging checklist

1. Run `go test -race`.
2. Minimize shared mutable state.
3. Replace ad-hoc state with channels or mutex.

G45. Table-driven testing template

```
func TestTwoSum(t *testing.T) {
    tests := []struct {
        name string
        nums []int
        target int
        want []int
    }{
        {"basic", []int{2, 7, 11, 15}, 9, []int{0, 1}},
    }
    for _, tt := range tests {
        t.Run(tt.name, func(t *testing.T) {
            got := twoSum(tt.nums, tt.target)
        })
    }
}
```

```
    if !reflect.DeepEqual(got, tt.want) {  
        t.Fatalf("got=%v want=%v", got, tt.want)  
    }  
})  
}  
}
```

Suggested Practice Order

1. G1-G8 for core map/slice fluency.
 2. G9-G16 for concurrency correctness.
 3. G23-G35 for DSA pattern coverage.
 4. G17-G22 and G36-G45 for production maturity.
-

Internet-Backed Reference Pointers

1. Go language spec: <https://go.dev/ref/spec>
2. Effective Go: https://go.dev/doc/effective_go
3. Go memory model: <https://go.dev/ref/mem>
4. Go concurrency patterns talk: <https://go.dev/talks/2012/concurrency.slide>
5. sync package docs: <https://pkg.go.dev/sync>
6. context package docs: <https://pkg.go.dev/context>
7. container/heap docs: <https://pkg.go.dev/container/heap>